Technical Appendices

Multilayer Graph Learning for Decreased Rank Attack Detection in RPL-Based IoT Networks

Sohail Abbas, Muhammad Kazim, Muhammad Fayaz, and Harun Pirim

Scope

This combined appendix PDF collects the five standalone technical appendices referenced from the main manuscript. Each appendix is also provided as a separate PDF in the repository so that readers can open the exact item cited in the paper.

Appendix List

- Appendix I: Cooja Trace Generation and Parsing.
- Appendix II: Mathematical Model and Training Protocol.
- Appendix III: Complete Detection Outputs.
- Appendix IV: Layer Analysis, Ablation, and Propagation Sensitivity.
- Appendix V: Reproduction Commands.

Appendix I
Cooja Trace Generation and Parsing

Sohail Abbas, Muhammad Kazim, Muhammad Fayaz, and Harun Pirim

1 Purpose of This Appendix

This appendix expands the simulation and parsing protocol used in the paper. The goal is to make clear what was obtained from Cooja/Contiki-NG, what was computed after parsing, and what was deliberately excluded from the model inputs. This matters because decreased-rank attack (DRA) detection can easily become inflated if an implementation-level attack log is accidentally used as a feature.

2 Cooja/Contiki-NG Scenario

The simulated network contains one RPL root and 49 non-root motes. The malicious-node ratios are 10%, 20%, and 30%, corresponding to 5, 10, and 15 DRA motes. For each ratio, five independent seeds are used. The seeds change attacker placement and network realization, so the held-out test seed evaluates generalization to an unseen topology and attacker placement rather than memorization of one trace.

Table 1: Trace-generation settings.

Setting	Value
Simulator	Cooja/Contiki-NG
Routing protocol	RPL
Network size	50 motes
Root	1 root, indexed as node 0
Attack type	Decreased-rank attack in DIO advertisements
Attack ratios	10%, 20%, 30%
Seeds per ratio	5
Simulation time	600 s
Warm-up removed	first 60 s
Observation window	60 s
Total logs	15
Graph snapshots	135
Node-level samples	6750

3 Trace-Driven Workflow

Figure 1 shows the complete trace-driven pipeline. The Cooja/Contiki-NG simulations generate serial logs under multiple DRA ratios and seeds. The parser converts these logs into fixed-width observation windows, extracts observable node and packet fields, and keeps DRA audit events separate. The flat-feature branch receives only the node-feature matrix, whereas graph models additionally receive the relation-specific adjacency matrices. This separation is important because it makes the comparison about representation structure rather than about access to different data sources.

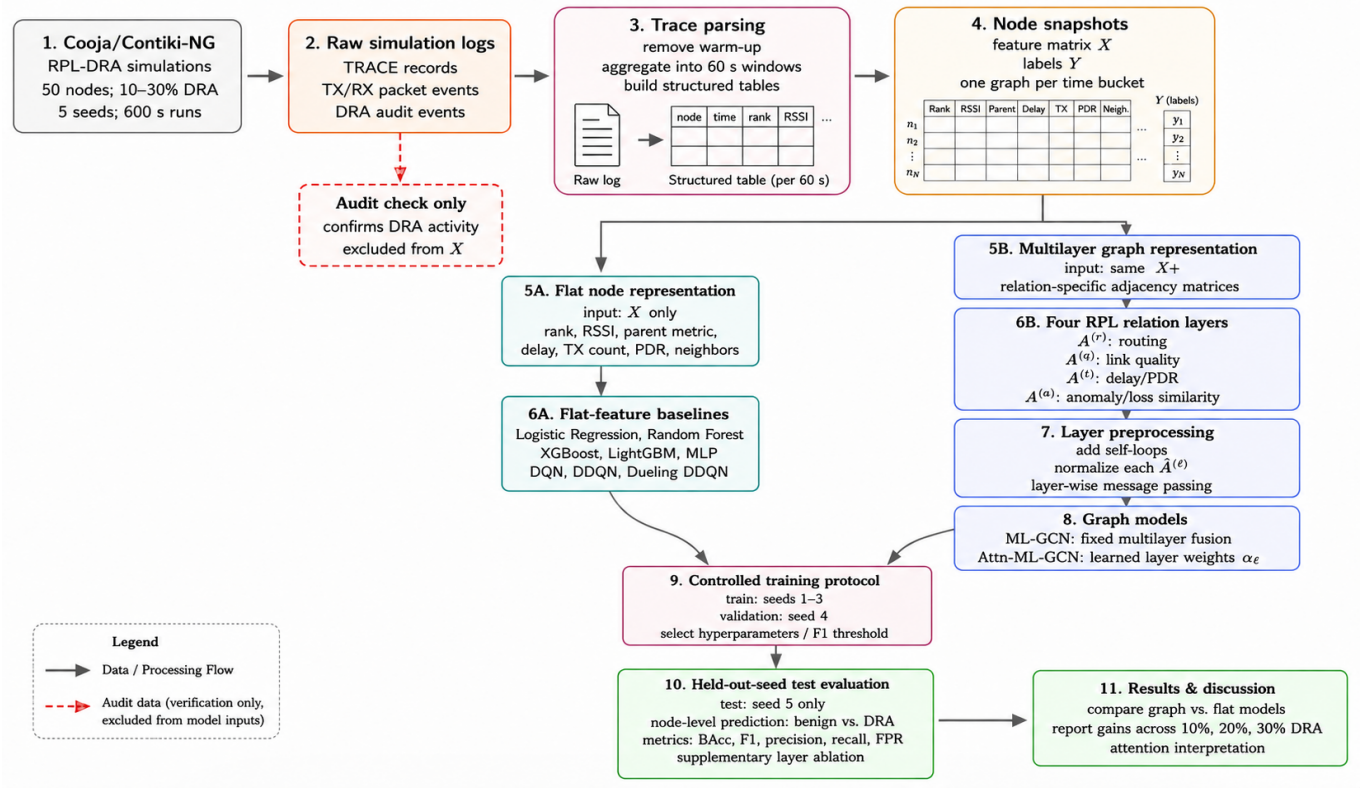

Figure 1: Trace-driven experimental workflow from Cooja/Contiki-NG simulation logs to flat-feature and multilayer graph model evaluation.

4 DRA Mechanism

Let R_i denote the actual RPL rank of node v_i and R_i^{adv} denote the rank that the node advertises in DIO control messages. Under benign operation, the advertised rank is equal to the actual rank:

$$R_i^{adv} = R_i. \tag{1}$$

For a malicious node $v_i \in \mathcal{M}$, the DRA implementation advertises an artificially improved rank:

$$R_i^{adv} = \max(R_{\min}, R_i - \delta_i), \qquad \delta_i > 0. \tag{2}$$

Here $R_{\min}$ prevents the advertised rank from becoming invalid, and δ_i is the malicious rank decrement. The attack is therefore not a direct packet-content attack. It manipulates the control-plane signal used by neighboring motes for parent selection.

A neighboring node v_j chooses its preferred parent from candidate neighbors $\mathcal{N}_j$. In simplified form, parent selection can be represented as

$$p(j) = \arg\min_{k \in \mathcal{N}_j} \Psi(R_k^{adv}, Q_{jk}), \tag{3}$$

where Q_{jk} summarizes link or parent cost. A DRA node can attract traffic because it reduces the first argument of the parent-selection objective without necessarily improving the true route quality.

5 Log Families Produced by the Simulation

The serial output is parsed into four separated artifact families:

1. **Scenario metadata**: attack ratio, seed, malicious-node set, and topology metadata.

2. **Periodic node traces**: node id, rank, DAG rank, preferred parent, parent metric, RSSI, neighbor count, role, and scenario label.

3. **Packet events**: transmission count, successful receptions, delivery delay, and packet-delivery ratio.

4. **DRA audit records**: implementation-level messages showing that a malicious node advertised a manipulated rank.

6 Feature Construction

For every node v_i in an observation window, the parser constructs the observable feature vector

$$x_i = [\tilde{i}, \tilde{R}_i, \tilde{G}_i, \tilde{S}_i, \tilde{M}_i, \tilde{N}_i, \tilde{D}_i, \tilde{T}_i, \tilde{P}_i], \tag{4}$$

where $\tilde{i}$ is the normalized node identifier, $\tilde{R}_i$ is rank, $\tilde{G}_i$ is DAG rank, $\tilde{S}_i$ is RSSI, $\tilde{M}_i$ is parent metric, $\tilde{N}_i$ is neighbor count, $\tilde{D}_i$ is mean delay, $\tilde{T}_i$ is the transmission count, and $\tilde{P}_i$ is PDR. These fields are observable from node and packet traces. The advertised-rank decrement, attack hook state, and internal audit flags are excluded.

7 Leakage Control

The DRA audit record is useful for validating that the scenario executed correctly, but it is not a legitimate detector input. If an audit field were included, the model would learn the simulator's attack marker rather than the network behavior caused by the attack. The pipeline therefore applies the following rule:

$$X = g(\text{node traces}, \text{packet events}), \qquad X \cap g(\text{DRA audit}) = \emptyset. \tag{5}$$

This rule is also applied to graph construction. Routing, link-quality, temporal, and trust/anomaly layers are built from observable routing and packet behavior, not from direct attack instrumentation.

8 Why the Held-Out-Seed Split Matters

A random split over node snapshots can put highly similar windows from the same topology into both training and testing. The paper avoids this by splitting at the Cooja-seed level: seeds 1–3 are used for training, seed 4 for validation, and seed 5 for final testing. The final test therefore contains unseen attacker placements and network realizations, which is a stricter check for publication than random sample splitting.

9 Repository Artifacts

The relevant repository folders are:

- `cooja/`: Contiki-NG/Cooja files and DRA simulation material.
- `scripts/parse_cooja_logs.py`: parser used to generate structured trace tables.
- `data/cooja_clean_5seed/`: parsed trace-derived benchmark data.
- `results/`: exported metrics and figures generated from the parsed snapshots.

Appendix II
Mathematical Model and Training Protocol

Sohail Abbas, Muhammad Kazim, Muhammad Fayaz, and Harun Pirim

1 Purpose of This Appendix

This appendix gives the detailed mathematical definitions supporting the main manuscript. It clarifies the multilayer graph representation, the relation-specific adjacency matrices, the GCN update, the attention fusion mechanism, the optimization objective, and the computational interpretation.

2 Multilayer RPL Graph

At each observation window, the parsed Cooja trace is converted into

$$\mathcal{G}_s = (V, \{A_s^{(\ell)}\}_{\ell \in \mathcal{L}}, X_s, Y_s), \qquad \mathcal{L} = \{r, q, t, a\}. \tag{1}$$

The set V contains the same physical motes in every relation layer. The four layers encode different evidence channels: routing (r), link-quality (q), temporal behavior (t), and trust/anomaly similarity (a). The feature matrix is $X_s \in \mathbb{R}^{N \times F}$ and the label vector is $Y_s \in \{0,1\}^N$.

The same physical node can be viewed as a set of layer replicas (v_i, ℓ). The conceptual supra-adjacency is

$$\mathcal{A}_{sup} = \begin{bmatrix} A^{(r)} & \lambda I & \lambda I & \lambda I \\ \lambda I & A^{(q)} & \lambda I & \lambda I \\ \lambda I & \lambda I & A^{(t)} & \lambda I \\ \lambda I & \lambda I & \lambda I & A^{(a)} \end{bmatrix}. \tag{2}$$

The implementation does not need to materialize this large block matrix. Instead, it computes relation-specific convolutions on the diagonal blocks and performs cross-layer coupling through fusion over the same node replicas. This makes the method easier to ablate because each RPL relation remains identifiable.

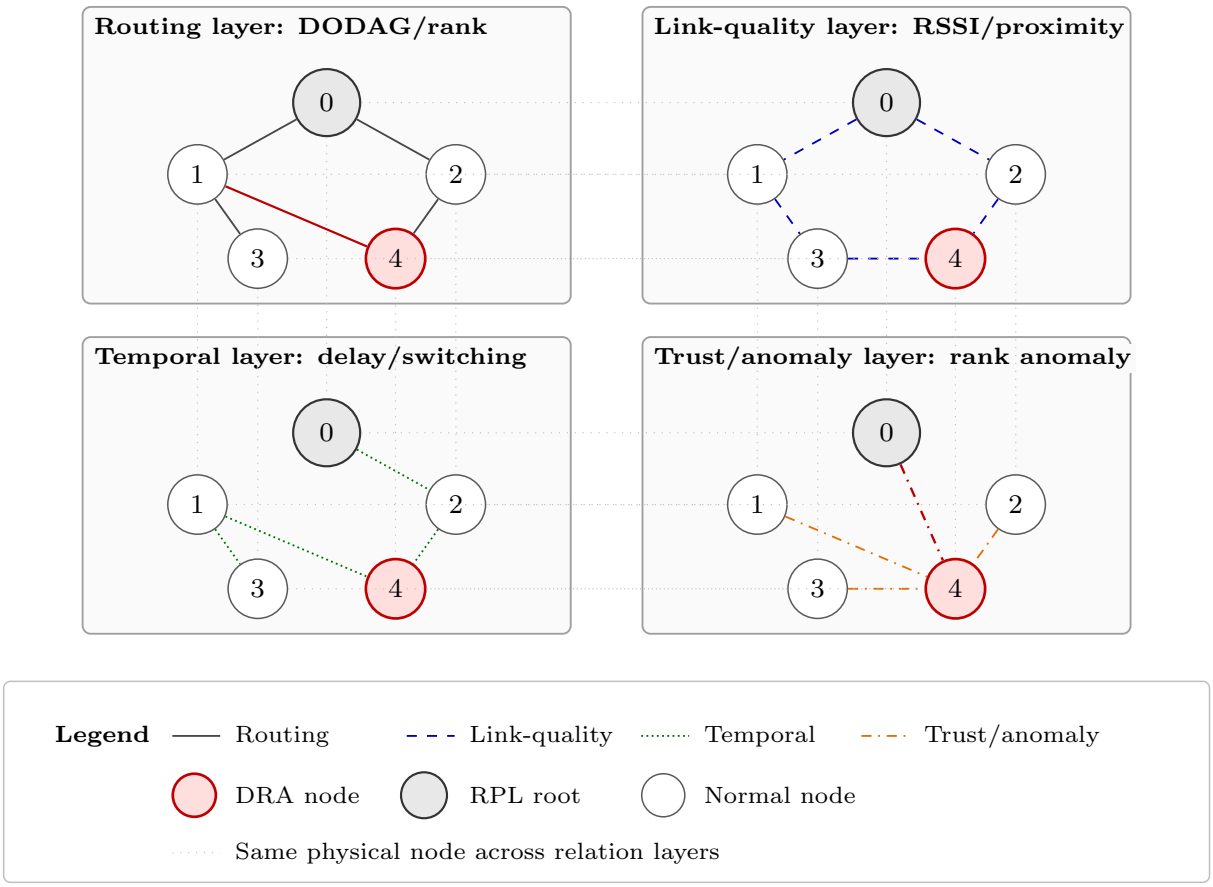

Figure 1: Conceptual multilayer RPL representation. The same physical nodes appear in routing, link-quality, temporal, and trust/anomaly layers, while the red node denotes the DRA mote. Dotted inter-layer links indicate that each layer contains a replica of the same physical mote.

Figure 1 illustrates the distinction between a multilayer RPL graph and a collapsed single graph. Each layer preserves a different edge meaning: DODAG parent-child support, link-quality consistency, temporal behavior, and anomaly similarity. The GCN branches in the proposed model process these relation-specific neighborhoods separately before fusion. This is the conceptual reason why the model can test whether preserving layer identity improves DRA detection.

3 Node Features

For node v_i, the observable feature vector is

$$x_i = [\tilde{i}, \tilde{R}_i, \tilde{G}_i, \tilde{S}_i, \tilde{M}_i, \tilde{N}_i, \tilde{D}_i, \tilde{T}_i, \tilde{P}_i], \tag{3}$$

where each component is standardized within the snapshot. Standardization avoids letting high-magnitude routing counters dominate lower-magnitude link and delay fields. No advertised-rank decrement or DRA audit hook is included in x_i.

4 Routing Layer

Let $p(i)$ be the preferred parent of node v_i. The routing layer is

$$A_{ij}^{(r)} = \begin{cases} 1, & p(i) = j \text{ or } p(j) = i, \\ 0, & \text{otherwise.} \end{cases} \tag{4}$$

This layer is important because DRA directly targets parent selection by making the attacker appear closer to the root.

5 Link-Quality Layer

The link-quality layer uses the same observed parent-child support but weights edges according to the parent metric:

$$A_{ij}^{(q)} = \begin{cases} (\max\{M_i/M_0, 1\})^{-1}, & p(i) = j \text{ or } p(j) = i, \\ 0, & \text{otherwise.} \end{cases} \tag{5}$$

Lower parent metric values imply stronger weights. This layer lets the model distinguish a structurally attractive parent from a reliable parent.

6 Temporal Layer

Let

$$z_i^{(t)} = [D_i, \Delta p_i, \Delta R_i], \tag{6}$$

where D_i is delay, Δp_i is parent-switching behavior, and ΔR_i is rank-change behavior in the observation window. Temporal similarity is

$$S_{ij}^{(t)} = \exp\left(-\frac{\|z_i^{(t)} - z_j^{(t)}\|_2^2}{\sigma_t^2}\right). \tag{7}$$

Edges are retained when this similarity is among the strongest temporal relations. This layer is not an attack label; it is a behavioral neighborhood showing which nodes exhibit similar dynamics.

7 Trust/Anomaly Layer

The trust/anomaly descriptor is

$$b_i = [a_i^{(R)}, a_i^{(M)}, 1 - P_i], \tag{8}$$

where $a_i^{(R)}$ is a low-rank anomaly score, $a_i^{(M)}$ is parent-metric deviation, and $1 - P_i$ is packet-loss behavior. The corresponding similarity is

$$A_{ij}^{(a)} = \exp\left(-\frac{\|b_i - b_j\|_2^2}{\sigma_a^2}\right). \tag{9}$$

This layer should be interpreted as suspicious-behavior similarity, not a direct maliciousness label. It gives the GCN a structured channel to compare nodes with similar abnormal patterns.

8 Layer-Specific GCN

For every layer ℓ, self-loops are added:

$$\tilde{A}^{(\ell)} = A^{(\ell)} + I. \tag{10}$$

The normalized adjacency is

$$\hat{A}^{(\ell)} = (\tilde{D}^{(\ell)})^{-1/2} \tilde{A}^{(\ell)} (\tilde{D}^{(\ell)})^{-1/2}, \qquad \tilde{D}_{ii}^{(\ell)} = \sum_j \tilde{A}_{ij}^{(\ell)}. \tag{11}$$

A one-hop relation-specific convolution is

$$H^{(\ell)} = \phi(\hat{A}^{(\ell)} X W^{(\ell)} + b^{(\ell)}). \tag{12}$$

The operation means that node v_i updates its representation by combining its own features with the features of neighbors connected under relation ℓ.

9 Attention Fusion

The non-attentive ML-GCN concatenates relation embeddings with fixed layer treatment. Attn-ML-GCN instead learns a scalar score θ_ℓ for each relation:

$$\alpha_\ell = \frac{\exp(\theta_\ell)}{\sum_{m\in\mathcal{L}} \exp(\theta_m)}. \tag{13}$$

The fused representation is

$$H = [\alpha_r H^{(r)} \| \alpha_q H^{(q)} \| \alpha_t H^{(t)} \| \alpha_a H^{(a)}]. \tag{14}$$

This formulation keeps the relation channels separated while still allowing the model to emphasize stronger evidence channels during training.

10 Node Classification and Loss

The classifier maps H_i to a two-class probability vector:

$$Z_i = \mathrm{softmax}(H_i W_o + b_o). \tag{15}$$

Weighted cross-entropy is used:

$$\mathcal{J}(\Theta) = -\sum_{i\in\mathcal{V}_{tr}} \sum_{c=0}^{1} \omega_c \mathbb{I}(y_i = c) \log Z_{i,c}. \tag{16}$$

The positive-class weight ω_1 compensates for the fact that malicious motes are the minority class at 10% and 20% attack ratios.

11 Threshold Selection

The model outputs probabilities, but the final detector needs a binary decision. The threshold τ is selected on validation seed 4:

$$\tau^\star = \arg\max_{\tau\in[0,1]} \mathrm{F1}_{val}(\tau). \tag{17}$$

The selected threshold is fixed before evaluating test seed 5. This avoids tuning on the test set.

12 Computational Cost

For sparse matrices, one forward pass has dominant cost

$$\mathcal{O}\left(\sum_{\ell\in\mathcal{L}} |E^{(\ell)}|F + |\mathcal{L}|NFd + N|\mathcal{L}|d\right). \tag{18}$$

The first term is sparse message passing, the second is feature projection, and the third is fusion. Attention adds only a small cost:

$$\mathcal{O}(|\mathcal{L}|Nd + |\mathcal{L}|). \tag{19}$$

13 Permutation Equivariance

For any node permutation matrix P, the model satisfies

$$f_\Theta(PX, \{P\hat{A}^{(\ell)}P^T\}_{\ell\in\mathcal{L}}) = P f_\Theta(X, \{\hat{A}^{(\ell)}\}_{\ell\in\mathcal{L}}). \tag{20}$$

Thus, predictions are tied to graph structure and features rather than arbitrary node numbering.

Appendix III
Complete Detection Outputs

Sohail Abbas, Muhammad Kazim, Muhammad Fayaz, and Harun Pirim

1 Purpose of This Appendix

This appendix reports the complete detection outputs supporting the main manuscript. It explains the metrics, reports the complete held-out-seed table, and discusses what the confusion matrix and precision–recall–FPR figure mean for DRA detection.

2 Metric Definitions

Let TP, TN, FP, and FN denote true positives, true negatives, false positives, and false negatives for malicious-node detection. The main metrics are

$$\text{Recall} = \frac{TP}{TP + FN}, \tag{1}$$

$$\text{Specificity} = \frac{TN}{TN + FP}, \tag{2}$$

$$\text{BAcc} = \frac{1}{2}(\text{Recall} + \text{Specificity}), \tag{3}$$

$$\text{Precision} = \frac{TP}{TP + FP}, \tag{4}$$

$$\text{F1} = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}, \tag{5}$$

$$\text{FPR} = \frac{FP}{FP + TN}. \tag{6}$$

Balanced accuracy is emphasized because raw accuracy can be misleading in imbalanced DRA detection: a detector can appear accurate by favoring the benign majority while missing attackers.

3 Complete Held-Out-Seed Results

Table 1: Held-out-seed test performance.

Model	BAcc	F1	Recall	Precision	FPR
Logistic Regression	0.668	0.465	0.489	0.443	0.154
Random Forest	0.662	0.443	0.611	0.347	0.287
XGBoost	0.671	0.468	0.507	0.435	0.165
LightGBM	0.613	0.381	0.370	0.392	0.144
MLP	0.665	0.437	0.759	0.307	0.429
DQN	0.606	0.383	0.707	0.263	0.496
DDQN	0.527	0.343	0.937	0.210	0.882
Dueling DDQN	0.664	0.434	0.789	0.300	0.461
Agg-GCN	0.621	0.395	0.489	0.331	0.247
ML-GCN	**0.721**	**0.531**	0.626	**0.460**	0.183
Attn-ML-GCN	0.689	0.489	0.567	0.430	0.188

4 Discussion of the Table

The best balanced accuracy and F1-score are obtained by ML-GCN. This means that preserving the four graph relation layers improves the attacker/benign tradeoff under the held-out seed. The improvement is not simply caused by using a neural model: the MLP and Q-network-style baselines use the same node features but do not receive adjacency matrices, and their F1-scores remain below ML-GCN.

Agg-GCN performs worse than ML-GCN because it collapses all relation layers into one support. This removes the difference between routing edges, link-quality edges, temporal similarity, and anomaly similarity. The result supports the manuscript's central claim: the relation identity of a multilayer RPL graph is useful, not merely the presence of more edges.

Attn-ML-GCN remains competitive but does not surpass ML-GCN in this held-out split. This is useful scientifically: the attention mechanism adds interpretability and adaptive layer weighting, but the fixed multilayer model is the strongest detector in the current Cooja setting. The paper therefore avoids overstating attention as universally superior.

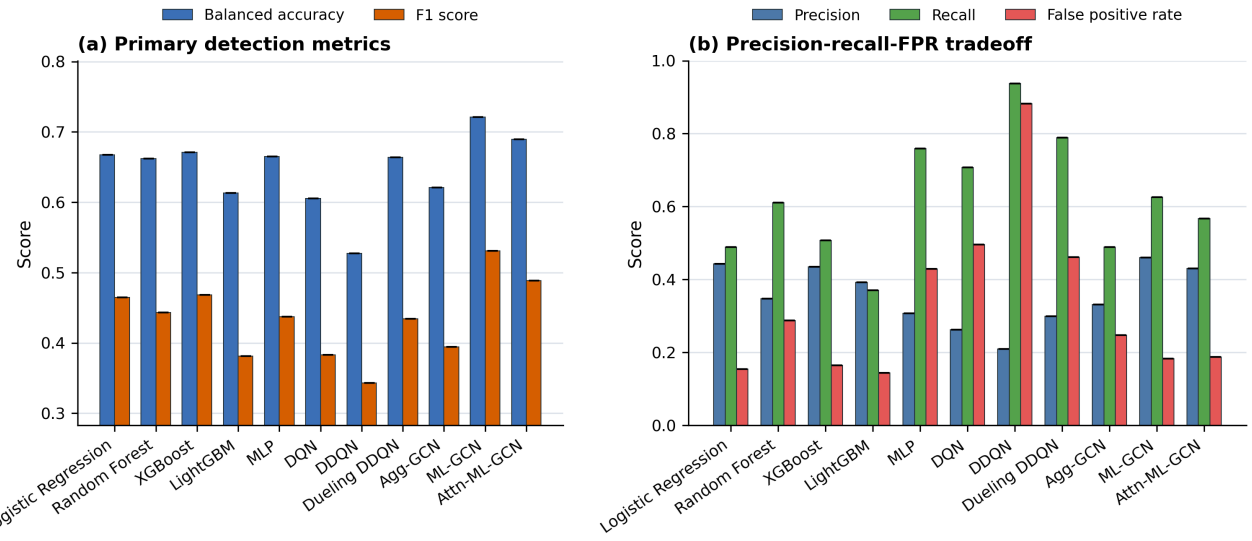

Figure 1: Overall model-family comparison across flat-feature, aggregate-graph, and multilayer graph models.

Figure 1 provides the broader visual comparison behind the main result table. The flat-feature models show different precision-recall tradeoffs, but the best operating point is obtained by the fixed-fusion multilayer graph model. The aggregate graph baseline does not match ML-GCN, supporting the interpretation that relation preservation matters more than simply increasing the number of graph edges.

5 Confusion Matrix

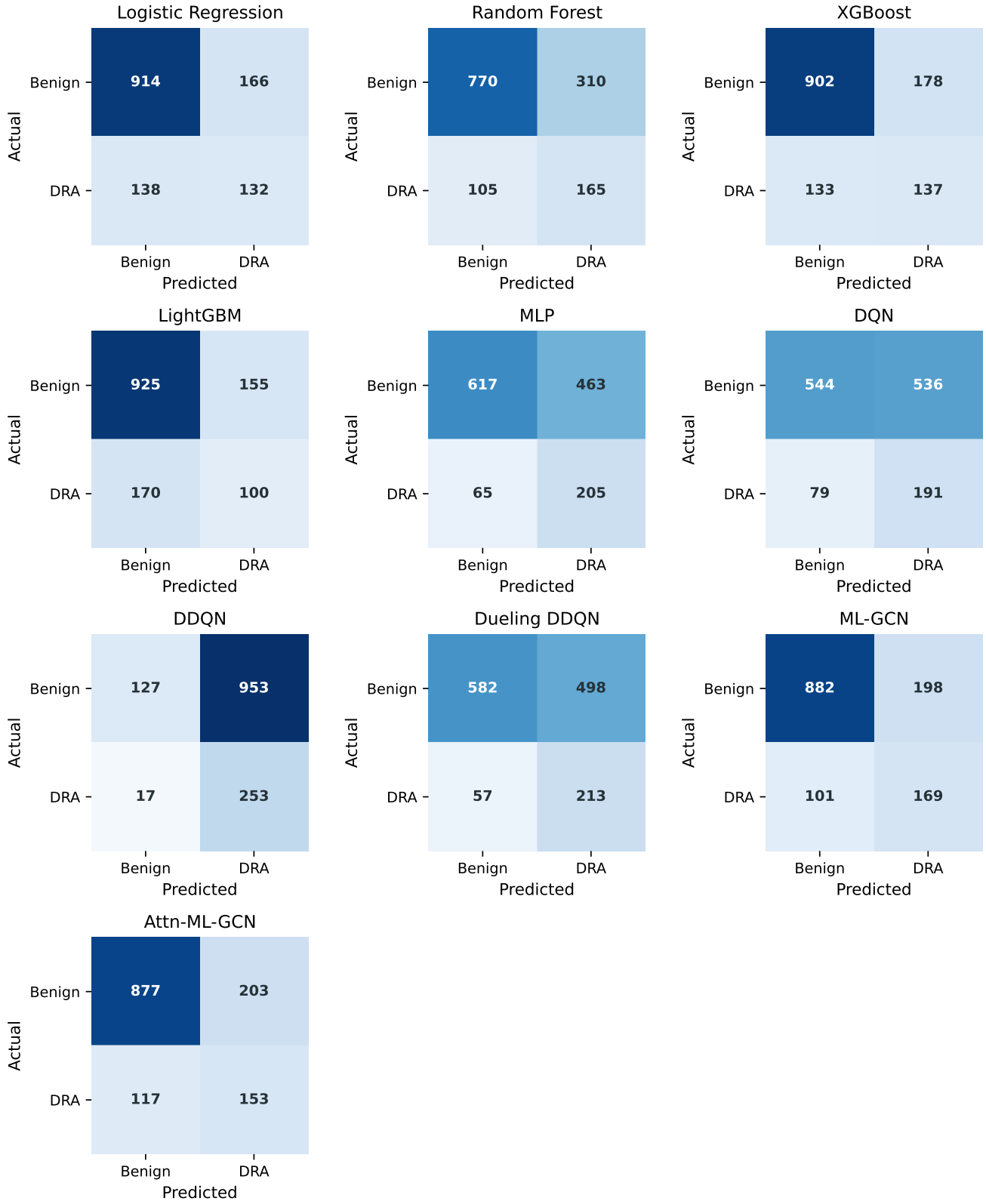

Figure 2: Confusion matrices for representative flat-feature, aggregate-graph, and multilayer graph models.

The confusion matrix shows whether a model obtains its score by detecting attackers, rejecting benign nodes, or over-triggering alarms. This is important for LLNs because false positives can cause unnecessary mitigation, parent reselection, or operator distrust. DDQN, for example, has high recall but a high FPR, meaning it catches many attackers while also marking many benign nodes as malicious. ML-GCN provides a more balanced behavior.

6 Precision, Recall, and FPR

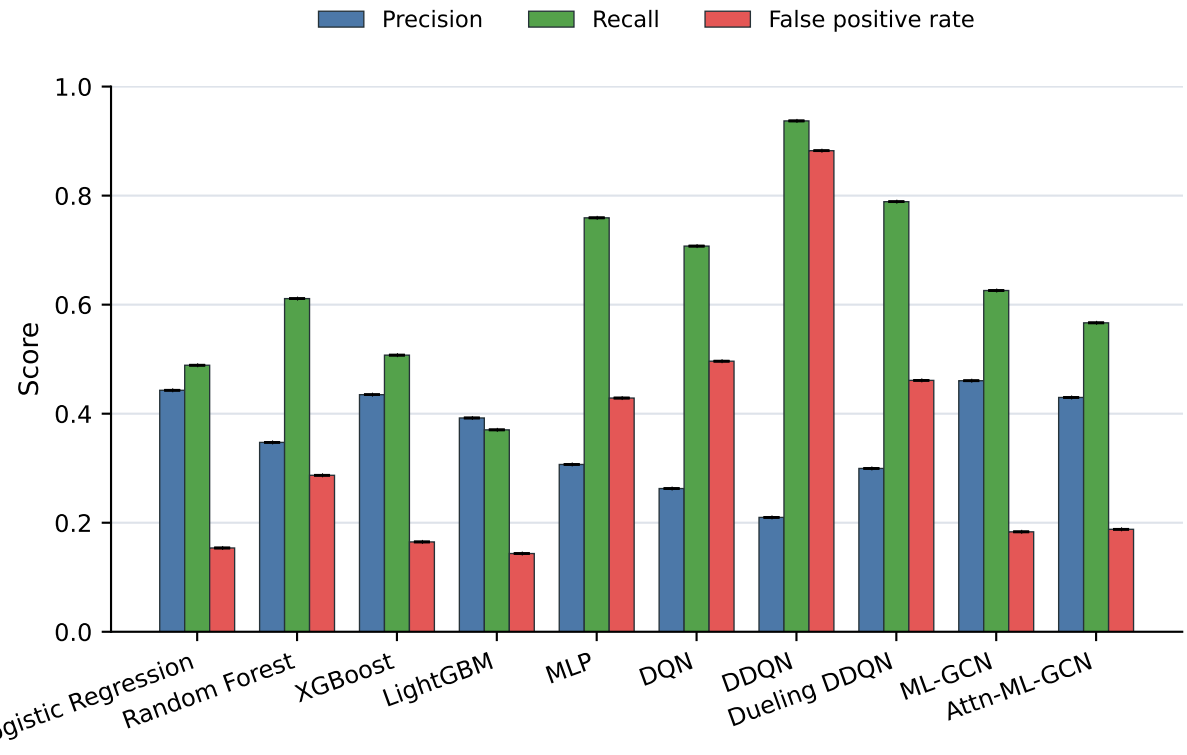

Figure 3: Precision, recall, and false-positive-rate comparison.

This figure separates the practical detection tradeoff. High recall alone is not enough because an LLN defense system must avoid excessive false alarms. The proposed graph model improves F1 because it maintains reasonable recall while controlling FPR better than many high-recall flat-feature neural baselines.

Appendix IV
Layer Analysis, Ablation, and Propagation Sensitivity

Sohail Abbas, Muhammad Kazim, Muhammad Fayaz, and Harun Pirim

1 Purpose of This Appendix

This appendix explains why the paper uses four relation layers and how each layer contributes to the final model. It expands the layer-density analysis, aggregation-support figure, ablation table, and propagation-depth sensitivity.

2 Layer Meaning

The four relation layers are designed to separate different mechanisms of RPL behavior:

- **Routing layer**: parent-child DODAG structure. This is the layer most directly affected by DRA because the attack changes parent-selection incentives.
- **Link-quality layer**: weighted version of the observed parent-child support. It distinguishes low-cost and high-cost parent relationships.
- **Temporal layer**: similarity in delay, packet-delivery behavior, and route dynamics across observation windows.
- **Trust/anomaly layer**: similarity in rank inconsistency, parent-metric deviation, and delivery loss.

3 Layer Structure

Table 1: Mean structural properties of the four RPL relation layers.

Layer	Mean edges	Density	Edge share
Routing	49.00	0.040	0.023
Link quality	49.00	0.040	0.023
Temporal	885.78	0.723	0.410
Trust/anomaly	1154.90	0.943	0.544

The routing and link-quality layers are sparse because each node has a limited parent-child structure. The temporal and trust/anomaly layers are denser because they connect nodes by similarity. If all edges are collapsed into one graph, the dense layers dominate the message-passing support and the model loses the distinction between physical routing relations and behavioral similarity relations.

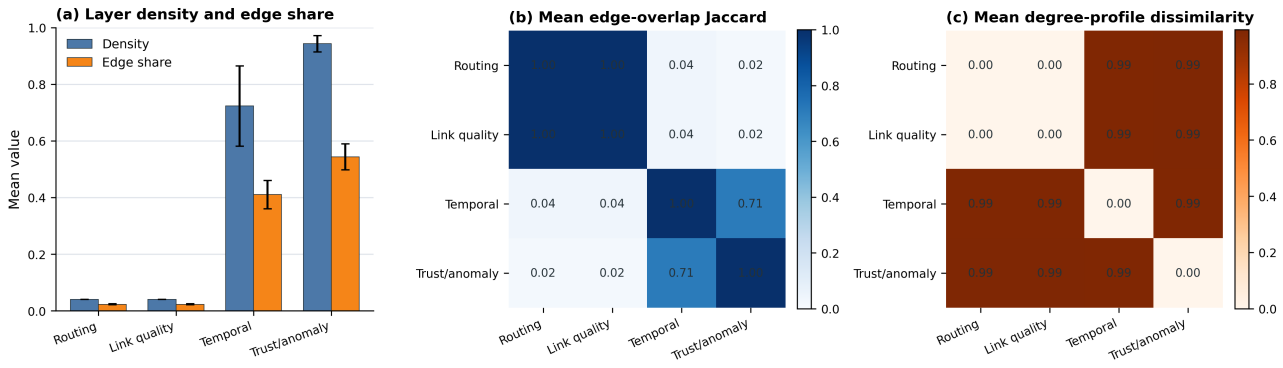

Figure 1: Structural properties of the relation layers, including layer support, density, and overlap patterns.

Figure 1 shows that the four layers are not interchangeable. Routing and link-quality remain sparse and protocol-local, while temporal and trust/anomaly relations are denser behavioral graphs. This structural asymmetry explains why aggregation can distort the learning problem: a single combined graph is dominated by dense behavioral edges and no longer preserves which relation generated each message-passing path.

4 Aggregation-Support Figure

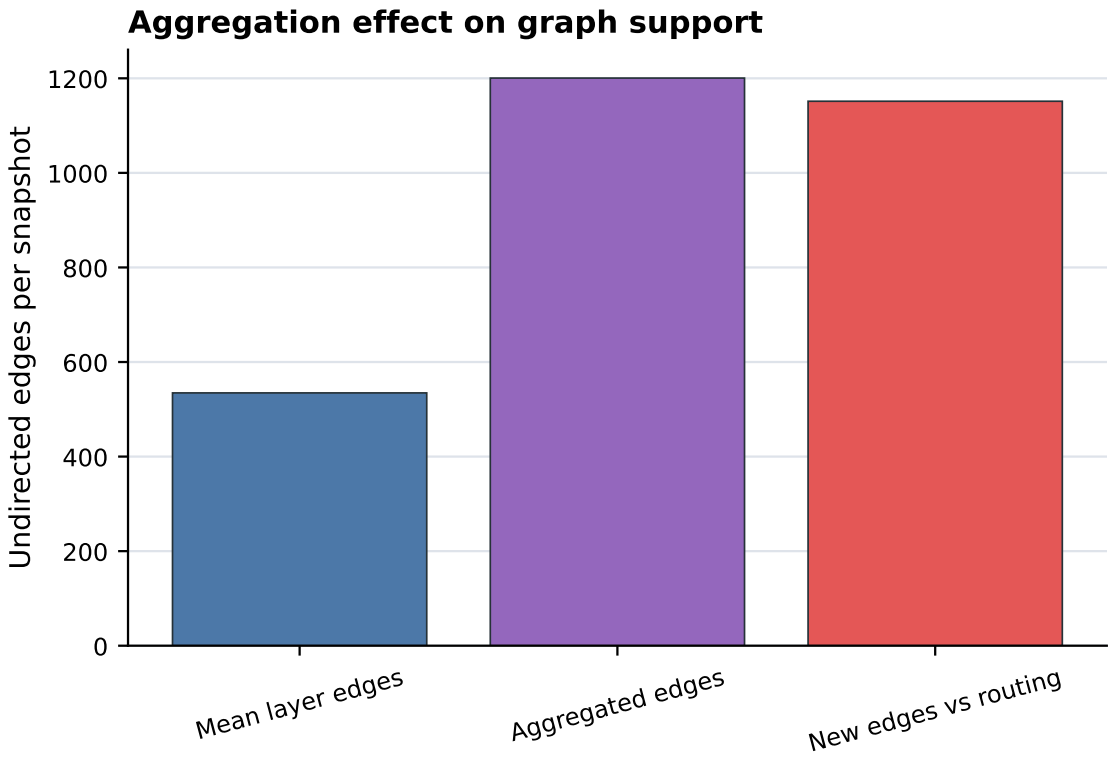

Figure 2: Aggregation-support analysis. Collapsing relation layers increases graph support and hides relation identity.

The figure explains why Agg-GCN is not equivalent to ML-GCN. Agg-GCN receives one combined support, so a message can come from a routing neighbor, a link-quality neighbor, or an anomaly-similar node without preserving which relation produced that edge. ML-GCN keeps the four channels separate and only fuses them after relation-specific convolution.

5 Layer Ablation

Table 2: Single-layer and leave-one-layer-out ML-GCN ablation.

Variant	BAcc	F1	FPR
All layers (ML-GCN)	**0.721**	**0.531**	0.183
Routing only	0.719	0.509	0.265
Link-quality only	0.706	0.495	0.262
Without temporal	0.693	0.481	0.258
Without routing	0.677	0.469	0.212
Without link quality	0.671	0.471	**0.150**
Temporal only	0.525	0.337	0.801
Trust/anomaly only	0.517	0.341	0.967

The ablation indicates that routing and link-quality information are highly informative for DRA detection, which is expected because decreased-rank attacks manipulate routing attractiveness. However, the full multilayer model obtains the best F1-score, showing that temporal and anomaly information add useful context when combined with the routing layers. The temporal-only and anomaly-only variants perform poorly because behavioral similarity alone is too broad and can connect benign nodes with similar noisy dynamics.

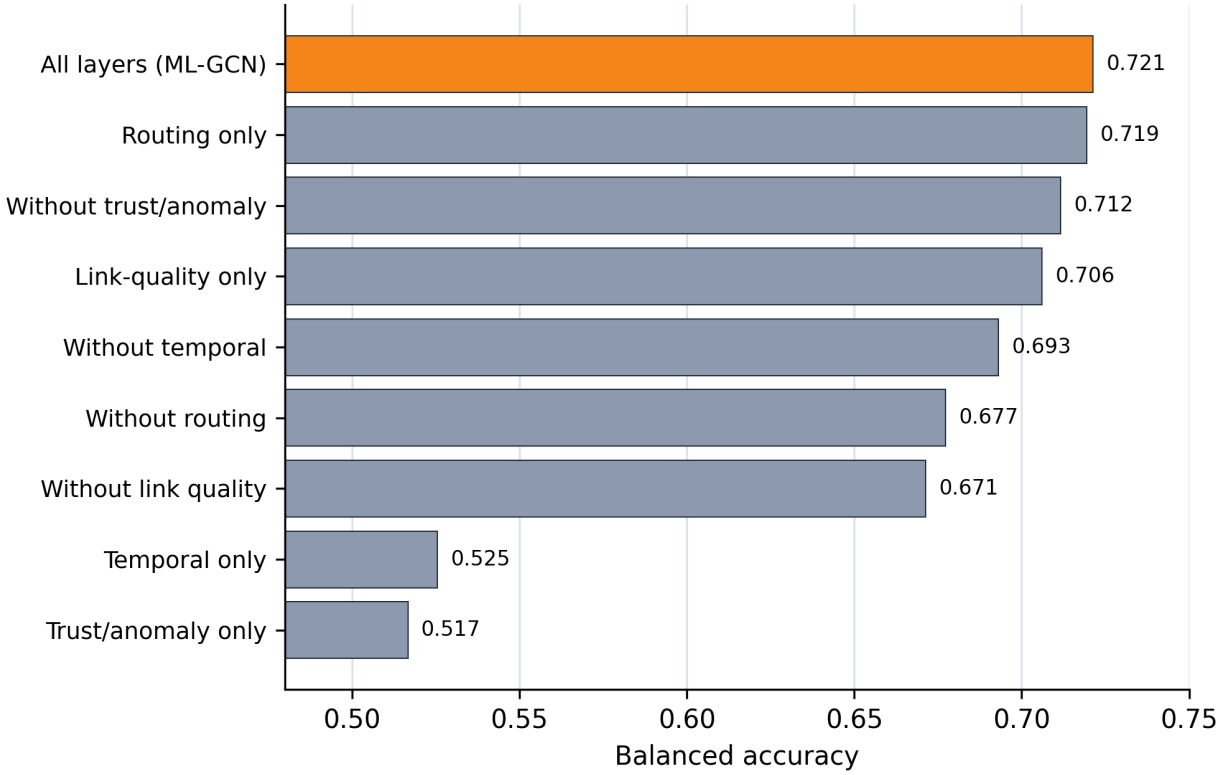

Figure 3: Layer-ablation visualization for ML-GCN under the held-out-seed protocol.

Figure 3 complements the ablation table by showing the tradeoff across detection metrics. The full multilayer model is preferred because it provides the best F1-score and balanced accuracy together. The single-layer variants reveal useful evidence channels, but none of them alone provides the same overall detection balance.

6 Propagation Sensitivity

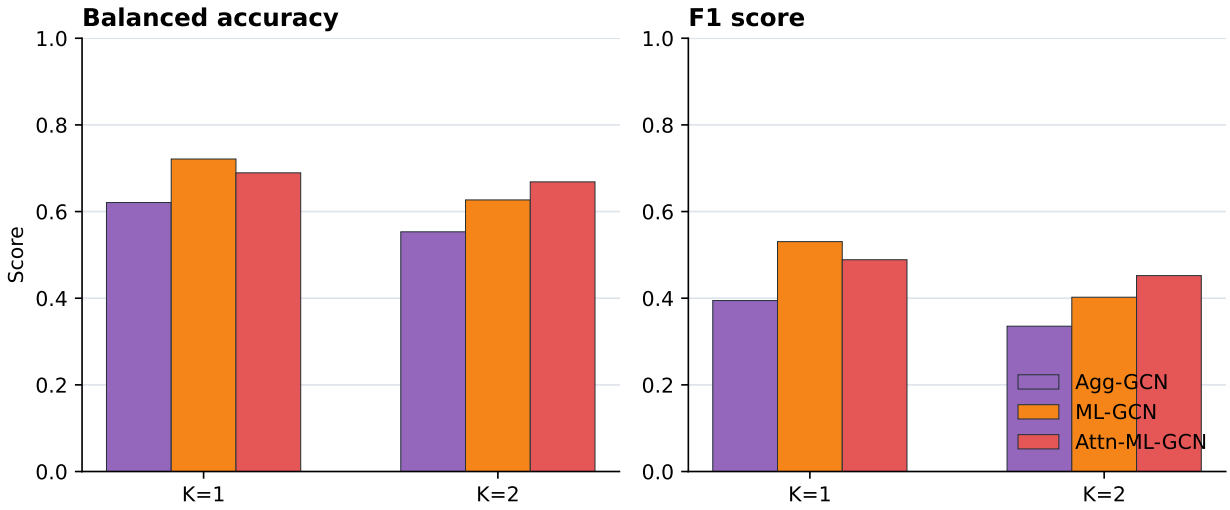

Figure 4: Propagation-depth sensitivity for graph-based models.

The sensitivity experiment compares one-hop and two-hop propagation. In this benchmark, two-hop propagation reduces the detection quality of the graph models. The reason is practical: the LLN snapshots are small, and DRA evidence can be localized around parent-child and nearby behavioral relations. Additional propagation mixes information from wider neighborhoods and can blur the difference between malicious and benign nodes. This supports the paper's choice of one-hop relation-specific message passing.

Appendix V
Reproduction Commands

Sohail Abbas, Muhammad Kazim, Muhammad Fayaz, and Harun Pirim

1 Purpose of This Appendix

This appendix provides the command-level reproduction path. It is intended for reviewers and readers who want to regenerate the parsed traces, model results, tables, and figures.

2 Environment Setup

```
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
```

The repository is organized so that Cooja files, parsers, training scripts, result exports, and plotting scripts remain separate. This makes it possible to rerun only the parser, only the model benchmark, or only the figure generation stage.

3 Parse Cooja Logs

```
python scripts/parse_cooja_logs.py \
  --log-root "/path/to/cooja-contiki-ng/examples/rpl-dra-ml/generated" \
  --output data/cooja_clean_5seed
```

The parser creates node traces, packet-event summaries, graph snapshots, and labels. DRA audit logs are retained for verification but excluded from model inputs.

4 Run the Full Model Benchmark

```
python scripts/run_cooja_experiment.py \
  --snapshots data/cooja_clean_5seed/node_snapshots.csv \
  --output results/cooja_clean_5seed_full_comparison \
  --models logistic_regression random_forest xgboost lightgbm mlp \
          dqn ddqn dueling_ddqn agg_gcn ml_gcn attn_ml_gcn \
  --epochs 350 \
  --hidden-dim 48 \
  --learning-rate 0.02 \
  --positive-class-weight 2 \
  --threshold-metric f1
```

The benchmark uses the same held-out-seed protocol as the paper: seeds 1–3 for training, seed 4 for validation, and seed 5 for final testing.

5 Run Layer Analysis

```
python scripts/analyze_layers.py \
  --snapshots data/cooja_clean_5seed/node_snapshots.csv \
  --output results/cooja_clean_5seed_layer_analysis
```

This command generates layer density, overlap, and aggregation-support artifacts used to explain why preserving relation identity matters.

6 Run Propagation Sensitivity

```
python scripts/run_graph_sensitivity.py \
  --snapshots data/cooja_clean_5seed/node_snapshots.csv \
  --output results/cooja_clean_5seed_graph_sensitivity \
  --steps 1,2
```

The sensitivity output supports the one-hop GCN design used in the manuscript.

7 Generate Tables and Figures

```
python scripts/make_tables.py results/cooja_clean_5seed_full_comparison
python scripts/make_figures.py results/cooja_clean_5seed_full_comparison
```

The generated figures are used directly in the manuscript and supplementary appendices.

8 Smoke Tests

```
pytest tests
```

The tests check parser behavior, graph construction, and key training utilities. They are not a substitute for rerunning the full Cooja benchmark, but they help verify that the repository environment is functioning.